

Отчёт по лабораторной работе

Файл: labA_01.cpp

Н.А. Пономарев, группа 25.M71-мм

13 января 2026 г.

Некоторые обозначения, используемые в данном тексте:

- $A_{m \times n} = (a_{ij})$ — матрица из m строк и n столбцов, а a_{ij} — элемент матрицы на строке i , столбце j .
- $\mathbb{1}_{m \times n}$ — матрица из единиц, соответствующего размера.
- $E_{n \times n}$ — единичная матрица.
- δ_{ij} — символ Кронекера.

1. Алгоритм

Алгоритм вычисляет значение некоторого матричного выражения. Все элементы матриц принадлежат полю вещественных чисел. На вход алгоритм получает матрицы $B_{n \times n}$, $C_{n \times n}$, и векторы $x_{n \times 1}$, $y_{n \times 1}$. В результате работы алгоритма получается матрица $A_{n \times n}$.

Первым делом вычисляется значение скаляра Ex по формуле:

$$Ex = \sum_{i=0}^{n-1} x_i = \mathbb{1}_{1 \times n} \cdot x_{n \times 1}.$$

Фактически считаем скалярное произведение.

Затем вычисляется вектор $CE_{n \times 1}$:

$$CE = C_{n \times n} \cdot \mathbb{1}_{n \times 1}$$

или поэлементно:

$$ce_{i1} = \sum_{k=0}^{n-1} c_{ik}.$$

Фактически получаем вектор сумм строк.

Далее вычисляем скаляр trace , который не является настоящим следом матрицы:

$$\text{trace} = \sum_{i=0}^{n-1} \sum_{k=0}^{n-1} b_{ik} \cdot c_{k1} = \mathbb{1}_{1 \times n} \cdot B_{n \times n} \cdot C E_{n \times 1}.$$

Затем вычисляем скаляр z_1 :

$$z_1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} b_{ij} \cdot E x \cdot y_i = E x \cdot (y^\top)_{1 \times n} \cdot B_{n \times n} \cdot \mathbb{1}_{n \times 1},$$

скаляр z_2 (просто скалярное произведение):

$$z_2 = \sum_{i=0}^{n-1} x_i y_i = (x^\top)_{1 \times n} \cdot y_{n \times 1},$$

и скаляр $z = z_1/z_2$.

В конце формируем матрицу $A_{n \times n}$:

$$A = \text{trace} \cdot C_{n \times n} + z \cdot \mathbb{1}_{n \times n} + E_{n \times n}$$

или поэлементно:

$$a_{ij} = \text{trace} \cdot c_{ij} + z + \delta_{ij}.$$

Обсуждение алгоритма. Сам по себе алгоритм большую часть времени занимается свёртками и выглядит довольно сложным для распараллеливания, поскольку мало независимых операций и вероятно большая нагрузка на работу с памятью.

2. Работа с кодом

В процессе выполнения работы код был модифицирован следующим образом:

- Исправлена синтаксическая ошибка в виде незакрытой фигурной скобки.
- Добавлен парсер аргументов командной строки.
- Слегка изменён вывод для упрощения постановки экспериментов.
- Декомпозирована логика вычислений для упрощения исследования и распараллеливания.
- Добавлен CMAKE.
- Применены исправления от CLANG-FORMAT и CLANG-TIDY.
- Написан скрипт для проведения замеров.

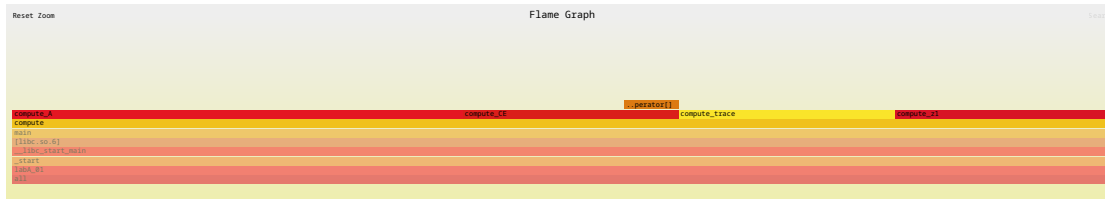


Рис. 1

3. Исследования hotspot'ов

С помощью утилит PERF и FLAMEGRAPH, был построен следующий график (см. рисунок 1), приближенный внутрь функции `compute` (картинка векторная, можно приблизить):

График, вероятно, подтверждает, что сложность алгоритма не в вычислениях, а в работе с памятью.

4. Распараллеливание

В соответствии с заданием, распараллеливание выполнялось с использованием двух технологий: OPENMP и MPI (OPENMPI).

Поскольку алгоритм состоит в основном из операций свёртки, то циклы второго уровня вложенности специально не распараллеливались, поскольку по эмпирическим оценкам время на создание потока больше, чем время работы вложенных циклов.

5. Исследование масштабируемости

Измерения проводились на системе с процессором Intel(R) Xeon(R) E-2136 CPU @ 3.30GHz с 12 потоками.

В случае с MPI измерения на кластере произвести не удалось, поэтому кластером служил тот же многоядерный процессор.

Измерения проводились при фиксированном $n = 10\,000$ и изменяющемся количестве потоков от 1 до максимальной степени двойки меньшей количеству потоков и с использованием всех потоков.

Перед запуском измерений при некотором фиксированном количестве ядер производилось 3 разогревочных запуска, затем производилось 10 измерений.

Итого, были получены результаты, изображенные на рисунке 2. Графики идентичны друг другу, отличие только в том, что на правом отсутствует график идеального распределения ради более детального анализа фактического ускорения.

Анализ ускорения. График показывает, что достичь идеального ускорения даже близко не получается. Более того, в сравнении с 4 потоками, на 8 и 12 потоках в случае OPENMP прирост почти не наблюдается, а в случае MPI происходит

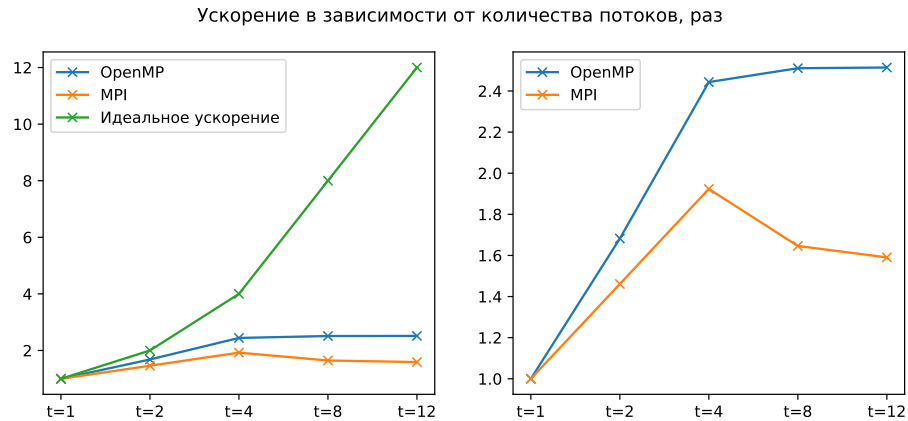


Рис. 2

замедление. Полагаю, что это связано с тем, что затраты на обмен данными и создание потоков начинают сравниваться с временем вычислений для OPENMP и превышают затраты на вычисления в случае MPI.

Выводы

В рамках лабораторной работы были выполнены следующие задачи:

1. Проанализирован алгоритм.
2. Улучшен код и создано окружение для сборки и измерений.
3. Проведено распараллеливание с помощью OPENMP и MPI (OPENMPI).
4. Проведены и проанализированы эксперименты.

К сожалению, большого ускорения достичь не удалось, что, вероятно, вызвано структурой самого алгоритма.

Исходный код доступен в репозитории: <https://github.com/WoWaster/spbu-hpc>.